

Projeto: Controle de Estoque

Objetivo: Criar um sistema de linha de comando para gerenciar o estoque de uma loja. O sistema não usará banco de dados nem dicionários; todas as informações devem ser armazenadas em uma única "lista de listas" na memória.

O estoque principal será uma lista. Cada produto será uma sublista dentro dela, obrigatoriamente nesta ordem de índices:

- Índice 0: Nome do Produto (Texto)
- Índice 1: Quantidade (Número Inteiro)
- Índice 2: Preço (Número Decimal)

Exemplo na prática: `estoque = [ ["Camiseta", 15, 49.90], ["Calça", 10, 89.90] ]`

O que vocês devem desenvolver: Vocês precisarão programar as funções abaixo e integrá-las a um menu interativo.

1. `adicionar_produto(estoque, nome, quantidade, preco):`

- Deve percorrer a lista para ver se o produto já existe.
- Se existir, some a quantidade nova à antiga.
- Se não existir, crie uma nova sublista e adicione ao estoque principal.

2. `realizar_venda(estoque, nome, quantidade):`

- Busque o produto pelo nome.
- Verifique se a quantidade em estoque é maior ou igual à solicitada.
- Se sim, deduza a quantidade e mostre o valor total a pagar. Se não, mostre um erro.

3. `exibir_estoque(estoque):`

- Mostre todos os produtos de forma organizada na tela.

Para que o sistema tenha o padrão de qualidade de um software real de mercado, haverá um desafio extra de pesquisa. Em ambientes profissionais, é essencial que o código seja previsível e fácil de ler por outros programadores. Em Python, uma das melhores formas de fazer isso é utilizando **anotações de tipos** (*type hinting*).

Como esse assunto não foi abordado em sala de aula, deverá ser feita uma pesquisa de como essa funcionalidade opera. Vocês devem descobrir como indicar na própria assinatura da função qual é o tipo de dado esperado para cada parâmetro (por exemplo, se o **nome** deve ser uma **str**, ou se a **quantidade** deve ser um **int**) e também como especificar qual será o tipo de retorno dessa função (ou indicar quando ela não tem retorno). Em um ambiente ideal, uma função teria o modelo a seguir:

`funcao(parametro: tipo) -> tipo_retorno:`